

MODULE 9 — Linux Concepts

Senior SWE Interview Track — Operating Systems

9.1 fork()

1. Why Interviewers Ask This

“What does fork return?” and “how many processes does this loop create?” are evergreen screens; the senior layer is COW mechanics and fork’s hazards in threaded/big processes.

2. Core Concept

`fork()` clones the calling process: child gets a copy of the address space (COW — Module 5.8), FD table (sharing open-file descriptions/offsets!), signal handlers, cwd, environment. Returns **twice**: child’s PID in the parent, **0 in the child**, -1 on failure. Only the calling thread exists in the child.

3. Internal Working

`fork` → `clone()` with child-copy flags → duplicate `task_struct`, copy page tables with all writable pages marked read-only (COW), bump file refcounts, copy signal table; child enters the run queue. Cost ∝ mapped memory (page-table copy), not RSS content.

4. ASCII Diagram

```
pid = fork();
    parent          child
pid = 1234 (child's pid)  pid = 0
same code, same line, COW-shared memory, SHARED file offsets

for (i=0;i<3;i++) fork();  -> 2^3 = 8 processes total (draw the tree!)
```

5. Real Production Example

Shells (fork+exec per command), Nginx/Postgres worker spawning, Redis BGSAVE (fork for COW snapshot), Unicorn/uWSGI prefork models, Android Zygote.

6. Advantages

Simple process creation with full context inheritance; COW makes it cheap; natural prefork server pattern; snapshot semantics.

7. Trade-offs

Threads don’t survive (locks held by dead threads → child deadlocks in malloc — the classic!); page-table copy cost for huge processes; COW spike risk; fork in a 60 GB process can fail on overcommit settings.

8. Common Mistakes

- Return-value confusion (0 = you’re the child; not “child gets nothing”).
- The fork-loop count: `n` forks in a loop → 2^n processes.
- Forgetting FD/offset sharing (parent and child interleave writes to the same log offset — actually fine for `O_APPEND`, corrupting otherwise).
- Calling non-async-signal-safe functions between fork and exec in threaded programs.

9. Performance Implications

fork of a small process ~100 μs; of a 60 GB-mapped process ~tens of ms (page tables) + COW faults after. `posix_spawn/vfork` avoid the copy when you’ll exec immediately.

10–11. Interview & Follow-ups

- “Output of this fork snippet?” (trace both branches; note buffered stdout duplication — `printf` without newline before fork prints twice!)
- “Why is fork dangerous in multithreaded programs?” “fork vs `vfork` vs `posix_spawn` vs `clone`?”

12. Coding/Debugging Scenario

Python service using multiprocessing (fork) hangs sporadically: a thread held a logging lock at fork time → child deadlocks; fix with `spawn` start-method or `fork-server`, or `pthread_atfork` handlers.

13. Best Practices

fork+exec immediately in threaded programs (or `posix_spawn`); flush stdio before fork; prefer spawn-type APIs in high-level languages.

14. Practice Questions

1. How many processes and what's printed: `fork() && fork() || fork();` — work it through.
2. Why does Redis recommend `vm.overcommit_memory=1`? (fork of huge process would fail conservative overcommit check)

9.2 exec()

1. Why Interviewers Ask This

fork/exec separation is *the* Unix design idea — interviewers test that you know exec **replaces** the image and why the split is powerful.

2. Core Concept

`exec*()` (`execve` and its wrappers `execl`/`execv`/`execvp`...) replaces the current process image with a new program: new code/data/heap/stack — **same PID**, same open FDs (minus `O_CLOEXEC` ones), same cwd/credentials (unless setuid binary). On success it **never returns**; code after exec runs only on failure.

3. Internal Working

`execve(path, argv, envp)`: kernel opens the binary, parses ELF (or shebang), tears down old VMAs, maps new segments (demand-paged), maps the dynamic linker if needed, resets signal handlers to default (ignored dispositions persist), preserves FD table, jumps to entry point. The fork/exec split is why redirection is trivial: *between* fork and exec, the child rearranges its own FDs (`dup2(logfd, 1)`) before exec — no API needed on exec itself.

4. ASCII Diagram

```
shell: ls > out.txt
fork() ----- child:
                  open("out.txt") -> fd 5
                  dup2(5, 1)         // stdout now the file
                  close(5)
                  execve("/bin/ls", ...) // image replaced, fd table kept
parent: wait(...)
Same PID before/after exec; new program inherits fds 0,1,2 as arranged.
```

5. Real Production Example

Every shell pipeline and container entrypoint; supervisors (systemd, runit) fork+exec services; CGI/subprocess APIs (`subprocess.run`, `ProcessBuilder`) are fork/(`vfork`/clone)+exec underneath.

6. Advantages

Orthogonality: process creation (fork) and program loading (exec) compose — redirection, pipes, environment setup all become child-side code between the two calls.

7. Trade-offs

Two syscalls where Windows has one (CreateProcess); fork's COW cost paid even when exec discards it (mitigations: vfork/posix_spawn); FD leaks into new programs without O_CLOEXEC (security issue).

8. Common Mistakes

- “exec creates a new process” — no; same process, new program.
- Writing code after exec expecting it to run (it runs only on failure — always perror+exit there).
- Forgetting FD inheritance → leaked sockets/db handles in child programs (set CLOEXEC).

9. Performance Implications

exec cost = ELF mapping + dynamic linking (demand paging defers most); large static binaries exec faster to first instruction than heavy dynamic linking; frequent short-lived fork+exec (shell-out per request!) is a real performance antipattern.

10–11. Interview & Follow-ups

- “How does the shell implement `a | b > f`?” (fork x2, pipe, dup2 choreography, exec)
- “What survives exec?” (PID, FDs sans CLOEXEC, cwd, umask, ignored signals) “What doesn't?” (memory, handlers, threads)

12. Coding/Debugging Scenario

A spawned child inherits the parent's listening socket → parent can't rebind after restart (“address in use”, ghost listener) → set SOCK_CLOEXEC.

13. Best Practices

O_CLOEXEC everywhere by default; check exec failure explicitly; use posix_spawn/subprocess APIs rather than raw fork+exec in threaded services.

14. Practice Questions

1. Implement `ls | grep foo` in C: pipe(), two forks, dup2s, execs, waits — write it.
2. Why do setuid binaries + exec need extra care with inherited FDs and environment? (privilege escalation surface)

9.3 wait() / waitpid()

1. Why Interviewers Ask This

wait is the reaping half of the process lifecycle — it's how you *prevent zombies*, and interviewers chain it directly into the zombie/orphan questions.

2. Core Concept

`wait(&status)` blocks until a child changes state (usually exits), returns its PID and status (exit code via `WEXITSTATUS`, killing signal via `WTERMSIG`). `waitpid(pid, &status, options)` targets a specific child; `WNOHANG` makes it non-blocking (poll for finished children). Reaping frees the child's PCB — until then it's a zombie.

3. Internal Working

Exited child → state Z, holds only PCB (exit status, rusage); kernel sends `SIGCHLD` to the parent. `wait*` finds a zombie child, copies status out, frees the task. If the parent ignores `SIGCHLD` explicitly (`SA_NOCLDWAIT`), children are auto-reaped.

4. ASCII Diagram

```
parent: fork -> child ... child exits -> [ZOMBIE: pcb only]
parent: wait(&status) -----> reaps: pcb freed
patterns:
  blocking:  pid = wait(&st);
  targeted:  waitpid(cpid, &st, 0);
  poll-all:  while ((p=waitpid(-1,&st,WNOHANG))>0) reap(p);  // in SIGCHLD handler
```

5. Real Production Example

Shells report exit codes via wait; systemd/supervisord reap and restart services; container init (tini/dumb-init) exists to reap orphaned zombies inside PID namespaces; CI runners harvest job exit statuses.

6. Advantages

Reliable exit-status delivery; resource cleanup; parent-driven lifecycle (restart policies, exit-code semantics).

7. Trade-offs

Blocking wait stalls the parent (use SIGCHLD+WNOHANG loop or a reaper thread); signal-based reaping has classic races (multiple children, coalesced SIGCHLD → must loop WNOHANG).

8. Common Mistakes

- Reaping one child per SIGCHLD (signals coalesce! loop until WNOHANG returns 0).
- Ignoring status macros (raw status int is encoded — use WIFEXITED/WEXITSTATUS).
- Believing kill -9 removes zombies (they're already dead; only reaping or parent death helps).

9. Performance Implications

Unreaped zombies leak PIDs → `fork: retry: Resource temporarily unavailable` at pid_max/threads-max; each zombie is tiny (KBs) but PID exhaustion halts the box.

10–11. Interview & Follow-ups

- “How do you run 100 children and collect all exit codes without blocking sequentially?”
- “What exactly does WNOHANG change?” “What happens if the parent never waits?” (zombies → next topic)

12. Coding/Debugging Scenario

Job runner accumulating thousands of Z processes: SIGCHLD handler reaps once per signal → convert to `while(waitpid(-1,&st,WNOHANG)>0)` loop; zombies drain.

13. Best Practices

Always reap; WNOHANG loops in handlers; in containers, run a real init (tini) as PID 1; propagate child exit codes meaningfully.

14. Practice Questions

1. Write the canonical SIGCHLD reaper (async-signal-safe: only waitpid + write).
2. Parent spawns children A(exit 0) and B(killed by SIGSEGV): show status decoding for both.

9.4 Zombie Process

1. Why Interviewers Ask This

Top-5 Linux interview question and a real incident type (“10,000 defunct processes”). Tests the exit-status handshake and containers’ PID-1 problem.

2. Core Concept

A zombie (Z, `<defunct>`) is a process that has **exited** but whose parent hasn't `wait()`ed yet. Only the PCB survives (PID, exit status, rusage) — no memory, no FDs, no CPU. It exists *by design*: the kernel must keep the exit status until the parent collects it.

3. Internal Working

exit → free memory/FDs → state Z → SIGCHLD to parent → parent `wait()` → PCB freed. If the parent exits first, the zombie is **reparented** (to `init`/subreaper) which reaps it. Zombies are un-killable (already dead; signals are meaningless). Fixing a live incident = make the *parent* reap: fix its code, or kill the parent so `init` inherits and reaps.

4. ASCII Diagram

```
child exits          parent wait()s
[running] --exit--> [ZOMBIE: pid+status only] --reap--> gone
                    ^ kill -9: NO EFFECT (already dead)

parent dies first:
[ZOMBIE] --reparent--> init/subreaper --auto-reap--> gone
Danger: buggy parent + spawn loop -> Z count grows -> PID exhaustion
```

5. Real Production Example

- Docker container whose entrypoint is your app as PID 1: your app never reaps grandchildren → zombies accumulate → solved by `--init`/tini (why that flag exists — great interview detail).
- CI agents/process-pool managers with broken SIGCHLD handling filling `ps` with `<defunct>` until fork fails fleet-wide.

6. Advantages

(Of the mechanism) Guarantees no exit status is ever lost; enables reliable supervision trees.

7. Trade-offs

Requires parental diligence; PID-space leak when neglected; confuses monitoring (process count alarms).

8. Common Mistakes

- “Zombies waste memory/CPU” — effectively neither; they waste **PIDs**.
- Trying `kill -9` on zombies; not knowing the kill-the-parent remediation.
- Confusing zombie (dead, unreaped) with orphan (alive, parentless) — next topic.

9. Performance Implications

Each zombie ≈ one `task_struct` (~KBs) + a PID; `kernel.pid_max/threads-max` exhaustion stops all process creation on the host — outage via bookkeeping.

10–11. Interview & Follow-ups

- “What is a defunct process and how do you get rid of it?” “Why can't you kill it?” “Why does `docker run --init` exist?”

12. Coding/Debugging Scenario

`ps aux | grep defunct | wc -l` = 30k, forks failing: identify the common PPID (`ps -eo ppid,stat | awk '$2~/Z/'` → count by ppid), fix or bounce that parent, add tini for containers.

13. Best Practices

Reap religiously (9.3 patterns); real `init` as PID 1 in containers; alert on Z-state counts and PID utilization.

14. Practice Questions

1. Write a 5-line C program that intentionally creates a zombie for 60 s; then modify to prevent it two different ways (`wait`; ignore SIGCHLD with `SA_NOCLDWAIT`).

2. Explain the full remediation ladder for 30k zombies with parent PID 812.

9.5 Orphan Process

1. Why Interviewers Ask This

The paired concept with zombies; also the mechanism behind daemonization and a real container gotcha (PID 1 semantics).

2. Core Concept

An orphan is a *live* process whose parent exited. The kernel **reparents** it to **init** (PID 1) — or the nearest **subreaper** (`prctl(PR_SET_CHILD_SUBREAPER)`, used by systemd/tmux/container inits). Init reaps it on exit, so orphans don't become permanent zombies. Orphans are normal and harmless per se — daemonization *deliberately* orphans (fork, parent exits, child continues under init).

3. Internal Working

Parent exit → kernel walks its children → reparent to subreaper-or-init → SIGCHLD flows there henceforth. Related: **orphaned process groups** and terminal control — background jobs may receive SIGHUP when the session leader dies (why `nohup`/`setsid`/`disown` exist). In containers, *your app* is PID 1: it inherits every orphan and must reap them (zombie link), and it doesn't get default signal handling (SIGTERM ignored unless you install a handler — the classic “container won't stop, Docker waits 10 s then SIGKILLs” bug).

4. ASCII Diagram

```

init(1)                                init(1)
  \                                     | \
  parent (exits!) ==>                 |  child (ORPHAN, alive, adopted)
    \                                 |
    child (running)                 SIGCHLD now goes to init; reaped on exit

daemonize: fork -> parent exits -> setsid() -> fork again -> daemon under init

```

5. Real Production Example

- SSH session dies → your long-running script gets SIGHUP and dies with it (unless `nohup`/tmux) — orphan/session semantics in action.
- Kubernetes pod: app as PID 1 ignoring SIGTERM → every deploy takes `terminationGracePeriod` and ends in SIGKILL → dropped in-flight requests. Fixed by proper signal handling or tini.

6. Advantages

Automatic adoption keeps the process tree consistent and guarantees eventual reaping; enables daemons and supervisors.

7. Trade-offs

Orphans escape their original supervision (restart logic loses them); accidental orphaning leaks “runaway” workers that keep consuming resources after their manager died.

8. Common Mistakes

- Zombie/orphan confusion (dead-unreaped vs alive-adopted) — have the one-line contrast ready.
- Not knowing subreapers exist (modern systemd/user session behavior).
- Assuming PID 1 in a container behaves like a normal process (signals! reaping!).

9. Performance Implications

Minimal by itself; the cost is operational — orphaned workers double-processing jobs, or holding locks/files their dead parent was supposed to manage.

10–11. Interview & Follow-ups

- “Parent dies before child — what happens? Who reaps?” “How does daemonization use orphaning?” “Why do containers need tini / signal handlers?”

12. Coding/Debugging Scenario

Deploy kills a job-manager but its 40 workers keep running (now under init), double-consuming queues → use process groups + kill(-pgid), cgroup-based teardown (systemd/K8s does this), or supervisor-managed PIDs.

13. Best Practices

Kill by process group/cgroup, not single PID; handle SIGTERM in anything that may run as PID 1; use subreapers for supervisors that spawn deep trees.

14. Practice Questions

1. Contrast zombie vs orphan across: alive?, PPID after event, who reaps, is it a problem?
2. Write the classic double-fork daemonization and explain each step's purpose.

9.6 Signals

1. Why Interviewers Ask This

Signals are the Unix control plane: kill, Ctrl-C, graceful shutdown, timeouts. Interviews probe SIGKILL vs SIGTERM, handler safety, and the kill-9-mythology.

2. Core Concept

Signals = asynchronous per-process (or per-thread-directed) notifications. Dispositions: default (terminate/core/ignore/stop), ignored, or **handled** (user function). Key set: SIGTERM (polite terminate, *catchable*), **SIGKILL (9)** and **SIGSTOP** (uncatchable, unblockable), SIGINT (Ctrl-C), SIGSEGV (fault), SIGCHLD (child state), SIGHUP (terminal death; repurposed as “reload config”), SIGPIPE (write to closed pipe — the silent killer of naive network code), SIGUSR1/2 (app-defined).

3. Internal Working

- Sender (**kill**, kernel fault, terminal driver) sets a bit in the target's pending mask; delivery happens at kernel → user transition (syscall return / interrupt return): kernel rigs the user stack to run the handler, then **sigreturn** restores context.
- Standard signals **don't queue** — 5 SIGCHLDs may deliver once (why reap loops!); real-time signals (SIGRTMIN+) queue with payloads.
- Per-thread masks (**pthread_sigmask**); process-wide handlers. Blocking syscalls may return **EINTR** when a signal arrives (handle-or-SA_RESTART — a classic subtle bug).
- Handler code must be **async-signal-safe** (no malloc/printf/locks!) — the professional pattern is the **self-pipe trick / signalfd / eventfd**: handler just writes one byte; the event loop does the real work.

4. ASCII Diagram

```
kill -TERM 1234 --> pending mask of pid 1234: {SIGTERM}
1234 returns from a syscall -> kernel: pending & ~blocked ?
-> push signal frame on user stack -> run handler() -> sigreturn -> resume
SIGKILL path: no handler possible -> kernel terminates directly
Graceful shutdown: SIGTERM -> handler sets flag/wakes loop ->
  drain requests -> exit(0); K8s: SIGTERM ... 30s ... SIGKILL
```

5. Real Production Example

Kubernetes/systemd stop sequence (SIGTERM → grace → SIGKILL) — every service you ship must drain on SIGTERM; Nginx `SIGHUP` reload / `SIGUSR1` log reopen; Go runtime turns SIGSEGV into panics and uses signals for preemption; JVM `kill -3` thread dumps.

6. Advantages

Universal, zero-dependency control channel; works on any process; kernel-enforced (SIGKILL always works — almost).

7. Trade-offs

Tiny bandwidth (a number, mostly); non-queuing loses events; handler safety is a minefield; races (signal between check and sleep) require `signalfd/pselect` patterns; not portable to Windows semantics.

8. Common Mistakes

- “kill = kill -9”: default `kill` sends SIGTERM (catchable, graceful); `-9` is the last resort that skips *all* cleanup (temp files, locks, in-flight writes).
- Believing SIGKILL kills anything: **D-state** processes and zombies don’t die (Module 1.6).
- `printf/malloc` in handlers; ignoring EINTR; forgetting SIGPIPE on socket writes (send with `MSG_NOSIGNAL` or ignore SIGPIPE).

9. Performance Implications

Signal delivery ≈ a forced user-context detour (~μs) — irrelevant at low rates; high-rate signal use (old-style timers) is an antipattern → `timerfd/eventfd`. EINTR storms under signal-heavy load surface as mysterious short reads/EINTR errors.

10–11. Interview & Follow-ups

- “SIGTERM vs SIGKILL — and design a graceful shutdown for your service.”
- “What can’t SIGKILL kill?” “What is async-signal-safety and how do real servers handle signals?” (self-pipe/`signalfd` into the event loop)

12. Coding/Debugging Scenario

Service drops requests on every deploy: no SIGTERM handler (or PID-1 default-ignore) → add handler: stop accepting, drain with deadline, exit; verify with `kill -TERM` locally.

13. Best Practices

Handle SIGTERM everywhere; handlers set flags/write `eventfd` only; `SA_RESTART` or EINTR loops around blocking calls; document a signal interface (reload, reopen, dump) for your daemons.

14. Practice Questions

1. Implement graceful shutdown for an `epoll` server via `signalfd` — sketch the loop changes.
2. Why may a process survive `kill -9`? Give both cases and the operator’s next steps.

9.7 Pipes

1. Why Interviewers Ask This

Pipes are IPC 101 plus a shell-mechanics probe (“how does `a | b` work?”) and the gateway to backpressure — a concept interviewers love to surface.

2. Core Concept

A pipe is a unidirectional in-kernel byte stream: `pipe(fds)` → `fds[0]` read end, `fds[1]` write end. Works between related processes (inherit FDs via fork). **Named pipes (FIFOs)** (`mkfifo`) get a filesystem name so unrelated processes can connect. Capacity: 64 KB default buffer.

3. Internal Working

Ring buffer in kernel memory. Semantics that matter: - Write to full pipe → **blocks** (backpressure! this is why `cmd1 | head` naturally throttles `cmd1`). - Read from empty pipe → blocks (or EAGAIN). - All write ends closed → read returns 0 (EOF) — how pipelines terminate. - All read ends closed → write gets **SIGPIPE**/EPIPE — how `head` kills upstream. - Writes ≤ **PIPE_BUF** (4 KB on Linux) are **atomic** (no interleaving between concurrent writers — key for shared log pipes). The shell builds `a | b`: `pipe()` → fork a (`dup2 write → stdout`) → fork b (`dup2 read → stdin`) → close unused ends everywhere (forgotten closes = the classic hang) → exec both.

4. ASCII Diagram

```
a's stdout -> [fd1 ===64KB kernel ring buffer=== fd0] -> b's stdin
full buffer -> a's write blocks (backpressure)
empty buffer -> b's read blocks
a exits (write ends closed) -> b reads EOF -> pipeline done
b exits first -> a gets SIGPIPE (default: dies) <- why `yes | head` stops
```

5. Real Production Example

Every shell pipeline and CI script; `docker logs`-style stdio plumbing; FIFOs for simple daemons' command channels; the pattern generalizes: bounded queues between microservices = pipes with the same backpressure math.

6. Advantages

Dead simple; automatic flow control (bounded buffer = built-in backpressure); composability (the Unix philosophy is literally this API); atomic small writes.

7. Trade-offs

Unidirectional (two pipes for duplex); byte stream (no message boundaries — you frame); related-process constraint (or FIFO); 64 KB buffer = deadlock fuel in naive parent-child stdout+stderr handling.

8. Common Mistakes

- The **subprocess deadlock**: parent reads child's stdout to EOF while child blocks writing a full stderr pipe (or vice versa) — both stuck. Fix: read both concurrently (threads/poll) or merge streams. Interviewers who've written CI systems *will* ask.
- Forgetting to close unused pipe ends after fork → readers never see EOF (pipeline hangs).
- Ignoring SIGPIPE in long-lived writers.

9. Performance Implications

Pipe throughput = memcpy speed (GB/s) with syscall-per-chunk overhead — size your reads/writes (64 KB chunks); `splice()` can move pipe ↔ socket/file data zero-copy. Latency ~μs (two syscalls + wakeup).

10–11. Interview & Follow-ups

- “Implement `ls | wc -l` in C.” “Why does `head` terminate the whole pipeline?” “What does PIPE_BUF atomicity give you?”
- “How is a pipe different from a socketpair / message queue?”

12. Coding/Debugging Scenario

CI step hangs forever spawning a chatty subprocess: stderr pipe full while parent reads only stdout → switch to `communicate()`-style concurrent drain; add output limits.

13. Best Practices

Close what you don't use immediately after fork; drain stdout+stderr concurrently; handle SIGPIPE/EPIPE; frame your messages or use `PIPE_BUF` records.

14. Practice Questions

1. Write two-process pipe code with all fd-closing done correctly; explain what breaks with each omitted close.
2. Design the backpressure story for a log shipper: what replaces “write blocks” when the pipe becomes a network queue?

9.8 Sockets

1. Why Interviewers Ask This

Sockets are the substrate of every backend system; interviews target the API lifecycle, TCP state subtleties (TIME_WAIT, backlog), and Unix domain sockets as the local IPC workhorse.

2. Core Concept

A socket is an FD representing a communication endpoint. Families: **AF_INET/6** (TCP/UDP over network) and **AF_UNIX** (same-host IPC through the kernel, addressed by filesystem path). Types: SOCK_STREAM (reliable byte stream), SOCK_DGRAM (messages). Server lifecycle: `socket` → `bind` → `listen` → `accept`; client: `socket` → `connect`; then `read/write`, `close`.

3. Internal Working

- `listen(fd, backlog)` creates two queues: SYN (half-open) and **accept queue** (completed handshakes waiting for `accept()`); overflowing the accept queue drops/rejects connections — the invisible outage cause (`ss -lnt` Rcv-Q vs Send-Q; `somaxconn`).
- Each connection = send/rcv kernel buffers; TCP flow control (rwnd) + congestion control decide pacing; `write` success ≠ delivered (only “queued in kernel”).
- Close choreography → **TIME_WAIT** (~60 s) on the side closing first: protects late packets; 30k ephemeral-port churn problems for clients that open/close rapidly (fix: connection pooling/keep-alive, not `tw_reuse` cargo-culting).
- **Unix domain sockets**: no protocol stack — kernel memcpy between buffers; ~2× TCP-loopback throughput, lower latency, and **FD passing** (SCM_RIGHTS) + peer credentials — the superpower TCP lacks.

4. ASCII Diagram

```
Server: socket->bind:8080->listen(backlog)      Client: socket->connect
        | SYN q -> [accept queue] <- 3-way handshake completes
accept() <---+ (queue full? drops!)
        -> new fd per connection -> epoll loop (Module 8)
write(fd) -> [send buf] --TCP--> [recv buf] -> read(fd)
close 1st -> FIN ... -> TIME_WAIT (60s, holds the 4-tuple)
AF_UNIX: client fd ==kernel memcpy==> server fd (no TCP/IP at all)
```

5. Real Production Example

Everything — plus the local flavor: nginx ↔ php-fpm/gunicorn via Unix sockets; Docker daemon (`/var/run/docker.sock`); Postgres local connections; sidecar proxies. TIME_WAIT port exhaustion is a rite-of-passage incident for high-QPS client services without pooling.

6. Advantages

Uniform FD semantics (epoll works — Module 8); location transparency (swap AF_UNIX ↔ AF_INET); rich control (TCP_NODELAY, SO_REUSEPORT, keepalive).

7. Trade-offs

Byte streams need framing (length-prefix/delimiters — “TCP is not a message protocol” trips juniors); lifecycle subtleties (half-close, RST vs FIN, CLOSE_WAIT leaks); buffers hide failure (write succeeded, peer dead).

8. Common Mistakes

- Assuming one `send` = one `recv` (fragmentation/coalescing — always frame).
- CLOSE_WAIT piles = *your* app didn’t close after peer’s FIN (FD leak signature, Module 7.6).
- Setting huge backlog but forgetting `net.core.somaxconn` caps it silently.
- Not knowing why Unix sockets beat localhost TCP (no stack, bigger effective copies, FD passing).

9. Performance Implications

Loopback TCP ~ GB/s with syscall overhead; AF_UNIX ~2× that; TCP_NODELAY vs Nagle for latency-sensitive small writes (40 ms delayed-ACK interactions — the classic p99 mystery); SO_REUSEPORT to spread accepts across workers.

10–11. Interview & Follow-ups

- “Walk through socket → accept → epoll for a web server.” “What is the backlog really?” “Why is TIME_WAIT there and when does it hurt?” “TCP vs Unix socket for two services on one host?”

12. Coding/Debugging Scenario

Load spikes cause silent connection failures though the app looks healthy: `ss -lnt` shows accept-queue overflow → raise backlog+somaxconn, accept faster (more workers), add SYN cookies check.

13. Best Practices

Frame messages explicitly; pool client connections; monitor TIME_WAIT/CLOSE_WAIT counts and accept-queue drops; prefer AF_UNIX for same-host RPC; tune NODELAY consciously.

14. Practice Questions

1. Implement a length-prefixed echo protocol over TCP — handle partial reads across frames.
2. A client fleet exhausts ephemeral ports at 50k conn/s churn — give three fixes ranked (pooling, keep-alive, more source IPs/tw tuning).
3. Demonstrate FD passing over a Unix socket — what does it enable architecturally? (zero-downtime listener handoff)

9.9 Shared Memory

1. Why Interviewers Ask This

The fastest IPC — and the one that reintroduces every synchronization problem from Module 3. Interviewers use it to test the “speed vs safety” IPC trade-off and modern usage (ring buffers, /dev/shm).

2. Core Concept

Two+ processes map the **same physical pages** into their address spaces: after setup, communication = memory access — zero syscalls, zero copies. APIs: POSIX `shm_open` + `mmap` (a file in tmpfs `/dev/shm`), `mmap(MAP_SHARED)` on a real file, System V `shmget/shmat` (legacy), `memfd_create` (anonymous, FD-passable).

3. Internal Working

`shm_open` creates a tmpfs object; `ftruncate` sizes it; each process `mmap`s it `MAP_SHARED` → page tables of both processes point at the same frames (Module 5 machinery). No kernel involvement per access — which means **no built-in synchronization**: you bring your own (process-shared mutexes `PTHREAD_PROCESS_SHARED`, unnamed semaphores in the segment, or lock-free rings on atomics). Robust mutexes (EOWNERDEAD) matter here: a crashed peer can die holding the lock.

4. ASCII Diagram

```

Process A page tables \                / Process B page tables
  vaddr 0x7f00... <----> [same physical frames] <----> vaddr 0x7f80...
                        /dev/shm/myseg (tmpfs)
Pattern: SPSC ring buffer in the segment:
[ head | tail | slots..... ] producer CAS/store tail, consumer head
Sync options: process-shared mutex+condvar | semaphores | lock-free atomics
Crash risk: peer dies holding lock -> robust mutex / sequence validation

```

5. Real Production Example

- Postgres shared buffers (all backends attach one segment); Chrome renderer ↔ GPU transport; aeron/LMAX-style shared-memory rings for trading (~100 ns latency); ML: model weights in /dev/shm shared across worker processes (PyTorch dataloaders); /dev/shm as scratch (it's RAM — counts against memory!).

6. Advantages

Fastest possible IPC (memory speed, no per-message syscalls/copies); natural for large objects (map once, share forever); works with lock-free structures for μs-to-ns messaging.

7. Trade-offs

You own all synchronization and memory layout (versioning! both sides must agree on struct layout/ABI); crash of one peer can leave corrupt state/held locks (need robust mutexes, sequence numbers, or restart-the-world); no message boundaries/flow control unless you build them; security = filesystem perms on the segment.

8. Common Mistakes

- Proposing shared memory without a synchronization story (instant Module-3 follow-up).
- Storing pointers in the segment (each process maps at different addresses — use offsets!).
- Forgetting /dev/shm consumes RAM and default-caps at 50% (tmpfs) — “disk full” on /dev/shm = memory issue.
- Ignoring cleanup: POSIX segments persist until shm_unlink even after all processes exit.

9. Performance Implications

Throughput = memory bandwidth; latency = cache-coherence (~100 ns cross-core) — 10–100× faster than sockets for large payloads. But false sharing/contention (Module 3.9) applies fully; NUMA placement of the segment matters on big boxes.

10–11. Interview & Follow-ups

- “Rank IPC mechanisms by speed and by safety; when is shared memory worth it?”
- “How do two processes synchronize access to a shared segment?” (process-shared mutex/robust; semaphores; lock-free) “What happens when one crashes mid-update?”

12. Coding/Debugging Scenario

Two services exchange 50 MB frames over localhost TCP at 30 fps → CPU burned on copies; move frames to a shared-memory ring + eventfd doorbell; CPU drops, latency 10×better. (The doorbell pattern — shm for data, fd for wakeup — is the pro answer.)

13. Best Practices

Use offsets not pointers; version your layout; pair shm (data plane) with a pipe/eventfd/socket (control plane + liveness); robust mutexes or lock-free with sequence checks; clean up segments on start (recover from crashes).

14. Practice Questions

1. Design a SPSC shared-memory ring buffer: layout, atomics/ordering, wrap handling, peer-crash recovery.
2. Compare passing a 1 GB dataset between processes via: pipe, Unix socket, TCP, shm, memfd+FD-passing — copies and syscalls for each.

Module 9 Cheat Sheet (one page)

Process lifecycle: fork (returns twice: child-pid/0; COW; FD-offset sharing) → exec (replace image, keep PID+FDs, never returns) → exit → **zombie** (dead, unreaped; PCB only; kill-proof) → wait (reap; WNOHANG loop for SIGCHLD) | parent dies → **orphan** (alive; adopted by init/subreaper). **Containers:** your app = PID 1 → must reap + handle SIGTERM (or use tini / `docker run --init`).

Signals: TERM catchable-graceful vs KILL/STOP uncatchable; D-state & zombies survive -9; handlers = async-signal-safe only (self-pipe/signalfd pattern); standard signals don't queue; EINTR; SIGPIPE on dead-peer writes.

IPC	Speed	Sync built-in	Boundaries	Killer feature / trap
Pipe/FIFO	GB/s, μ s	blocking = backpressure	byte stream	PIPE_BUF atomic $\leq 4K$; stdout/stderr drain deadlock
Unix socket	$\sim 2\times$ loopback TCP	flow control	stream or dgram	FD passing (SCM_RIGHTS), peer creds
TCP socket	network	flow+congestion ctl	byte stream (frame it!)	backlog/accept-queue; TIME_WAIT; CLOSE_WAIT leaks
Shared memory	memory speed	none — BYO	none	offsets not pointers; robust mutex; /dev/shm = RAM

Shell mechanics: `a | b > f` = pipe + fork $\times 2$ + dup2 + close-unused + exec; `head` kills upstream via SIGPIPE; EOF when all write ends close.

Top Interview Questions

1. fork return values; how many processes does this loop create; fork+threads hazard.
2. How does the shell implement pipelines and redirection? (write the C)
3. Zombie vs orphan — definitions, causes, fixes, container PID-1 angle.
4. SIGTERM vs SIGKILL; design graceful shutdown; what survives kill -9.
5. Why won't my container stop / why is it full of `<defunct>`? (tini)
6. TCP accept queue/backlog; TIME_WAIT vs CLOSE_WAIT diagnosis.
7. Rank IPC options for a given workload; shared memory + its sync story.

Common Mistakes (module-wide)

- Fork-count errors; expecting code after exec to run; unflushed stdio double-printing.
- One-reap-per-SIGCHLD; kill -9 as zombie cure; printf in signal handlers.
- Unclosed pipe ends (no EOF); single-stream subprocess draining (deadlock).
- Treating TCP as message-oriented; ignoring EINTR/SIGPIPE.
- Shared memory with raw pointers or no crash-recovery plan.

Mock Interview (self-test, ~25 min)

1. (Code) Implement `ps aux | grep java | wc -l` in C — full fd choreography, waits, and error paths.
2. (Prod) Host has 28k `<defunct>` processes, forks failing. Diagnose (find common PPID), remediate now, prevent forever (reaper patterns, tini).
3. (Design) Two same-host services must exchange 100k msgs/s of 200 B and occasionally 500 MB blobs. Pick IPC per path (socket control plane + shm data plane) and defend the sync/crash story.

4. (Depth) `kill -TERM <pid>` — trace everything from your shell to the target's handler running, including masks, pending sets, and delivery points.
5. (Trap) “Our app is PID 1 in the container; SIGTERM works in dev (not containerized) but the pod always takes 30 s to die.” Explain precisely and fix two ways.